LAB REPORT
Finite State Machines

3220102382　潘谷雨

2024.12.12

1. Problem Description

(1)virtual finite state machine

We wish to implement a finite state machine (FSM) that recognizes two specific sequences of applied input symbols, namely four consecutive 1s or four consecutive 0s. There is an input w and an output z. Whenever w =1 or w =0 for four consecutive clock pulses the value of z has to be 1;otherwise, z = 0. Overlapping sequences are allowed, so that if w =1 for five consecutive clock pulses the output z will be equal to 1 after the fourth and fifth pulses. Figure 71 illustrates the required relationship between w and z.

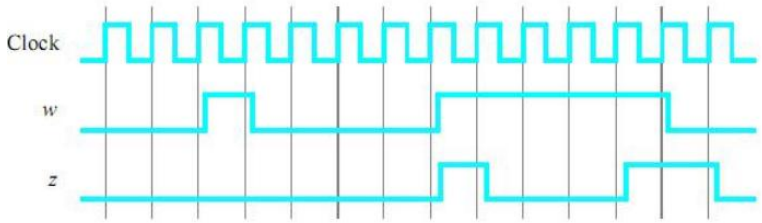


Figure 71. Required timing for the output z.

A state diagram for this FSM is shown in Figure 72. For this part you are to manually derive an FSM circuit that implements this state diagram, including the logic expressions that feed each of the state flip-flops. To implement the FSM use nine state flip-flops called y8,...,y0 and the one-hot state assignment given in Table 4.

Name	State Code $y_8y_7y_6y_5y_4y_3y_2y_1y_0$
A	000000001
B	000000010
C	000000100
D	000001000
E	000010000
F	000100000
G	001000000
H	010000000
I	100000000

Table 4. One-hot codes for the FSM.

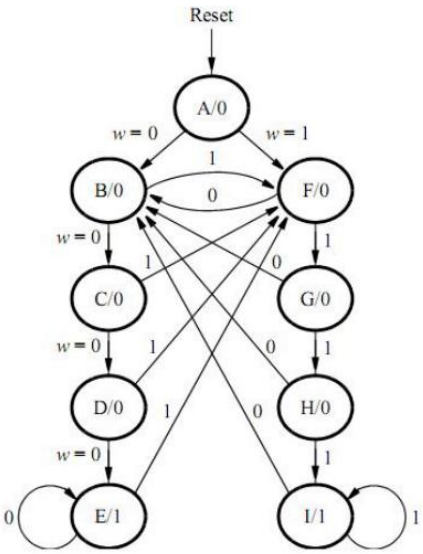


Figure 72. A state diagram for the FSM.

Design and implement your circuit on the DE2 board as follows.

1. Create a new Quartus II project for the FSM circuit. Select as the target chip the Cyclone II EP2C35F672C6,which is the FPGA chip on the Altera DE2 board.
2. Write a VHDL file that instantiates the nine flip-flops in the circuit and which specifies the logic expressions that drive the flip-flop input ports. Use only simple assignment statements in your VHDL code to specify the logic feeding the flip-flops. Note that the one-hot code enables you to derive these expressions by inspection. Use the toggle switch SW0 on the Altera DE2 board as an active-low synchronous reset input for the FSM, use SW1 as the w input, and the pushbutton KEY0 as the clock input which is applied manually. Use the green LED LEDG0 as the output z, and assign the state flip-flop outputs to the red LEDs LEDR8 to LEDR0.
3. Include the VHDL file in your project, and assign the pins on the FPGA to connect to the switches and the LEDs, as indicated in the User Manual for the DE2 board. Compile the circuit.
4. Simulate the behavior of your circuit.
5. Once you are confident that the circuit works properly as a result of your simulation, download the circuit into the FPGA chip. Test the functionality of your design by applying the input sequences and observing the output LEDs. Make sure that the FSM properly transitions between states as displayed on the red LEDs, and that it produces the correct output values on LEDG0.
6. Finally, consider a modification of the one-hot code given in Table 4. When an FSM is going to be implemented in an FPGA, the circuit can often be simplified if all flip-flop outputs are 0 when the FSM is in the reset state. This approach is preferable because the FPGA's flip-flops usually include a clear input port, which can be conveniently used to realize the reset state, but the flip-flops often do not include a set input port.

Table 5 shows a modified one-hot state assignment in which the reset state, A, uses all 0s. This is accomplished by inverting the state variable y0. Create a modified version of your VHDL code that implements this state assignment.

Hint: you should need to make very few changes to the logic expressions in your circuit to implement the modified codes. Compile your new circuit and test it both through simulation and by downloading it onto the DE2 board.

Name	State Code $y_8y_7y_6y_5y_4y_3y_2y_1y_0$
A	000000000
B	000000011
C	000000101
D	000001001
E	000010001
F	000100001
G	001000001
H	010000001
I	100000001

Table 5. Modified one-hot codes for the FSM.

(2)finite state machine

For this part you are to write another style of VHDL code for the FSM in Figure 72. In this version of the code you should not manually derive the logic expressions needed for each state flip-flop. Instead, describe the state table for the FSM by using a VHDL CASE statement in a PROCESS block, and use another PROCESS block to instantiate the state flip-flops. You can use a third PROCESS block or simple assignment statements to specify the output z.

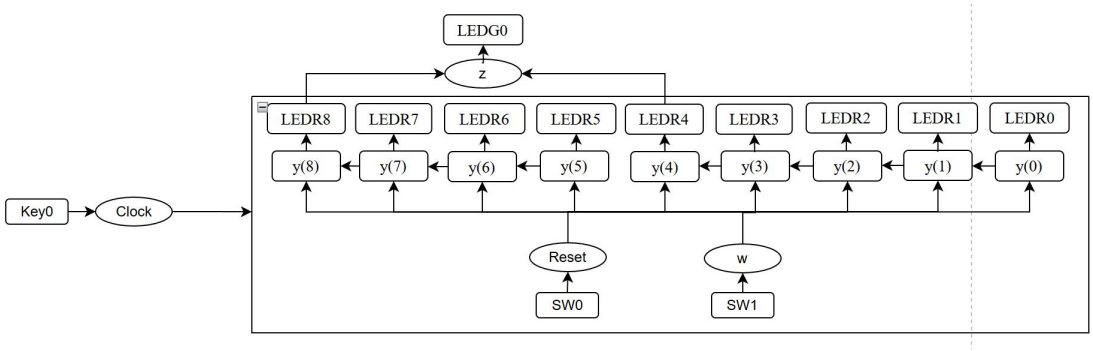
Implement your circuit as follows.

1. Create a new project for the FSM. Select as the target chip the Cyclone II EP2C70F896C6.
2. Include in the project your VHDL file that uses the style of code as given. Use the toggle switch SW0 on the Altera DE2 board as an active-low synchronous reset input for the FSM, use SW1 as the w input, and the pushbutton KEY0 as the clock input which is applied manually. Use the green LED LEDG0 as the output z,and use nine red LEDs, LEDR8 to LEDR0, to indicate the present state of the FSM. Assign the pins on the FPGA to connect to the switches and the LEDs, as indicated in the User Manual for the DE2 board.
3. Before compiling your code it is possible to tell the Synthesis tool in Quartus II what style of state as-signment it should use. Choose Assignments > Settings in Quartus II, and then click on the Analysis and Synthesis item on the left side of the window. Change the parameter State Machine Processing to the setting Minimal Bits.
4. To examine the circuit produced by Quartus II open the RTL Viewer tool. Double-click on the box shown in the circuit that represents the finite state machine, and determine whether the state diagram that it shows properly corresponds to the one in Figure 72. To see the state codes used for your FSM, open the Compilation Report, select the Analysis and Synthesis section of the report, and click on State Machines.
5. Simulate the behavior of your circuit.
6. Once you are confident that the circuit works properly as a result of your simulation, download the circuit into the FPGA chip. Test the functionality of your design by applying the input sequences and observing the output LEDs. Make sure that the FSM properly transitions between states as displayed on the red LEDs, and that it produces the correct output values on LEDG0.
7. In step 3 you instructed the Quartus II Synthesis tool to use the state assignment given in your VHDL code. To see the result of changing this setting, open again the Quartus II settings window by choosing Assignments > Settings, and click on the Analysis and Synthesis item. Change the setting for State Machine Processing from Minimal Bits to One-Hot. Recompile the circuit and then open

the report file, select the Analysis and Synthesis section of the report, and click on State Machines. Compare the state codes shown to those given in Table 5, and discuss any differences that you observe.

2. Design Formulation

(1)virtual finite state machine

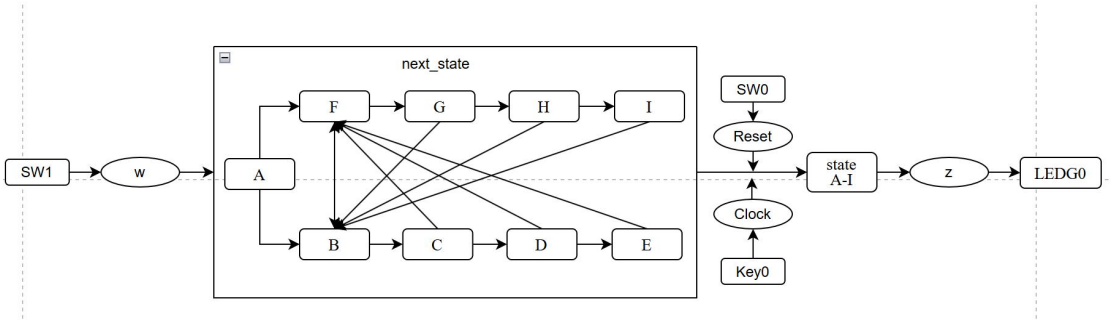


Pic.1.1 design formulation of virtual finite state machine

y(8) to y(0) are status registers, each storing one bit of state information. Together, these registers represent the current state of the FSM and is determined based on the current state and an input signal w.

SW0 is connected to the reset input and is used to initialize the FSM to the start state; Key0 is connected to the Clock input and is used to drive the state transition of the FSM; and SW1 is connected to the input w, which is used as a condition input to trigger a state transition.

(2)finite state machine



Pic.2.1 design formulation of finite state machine

States A-I are state registers used to store the current state of the FSM. The state transition logic determines the next state based on the current state and the input w. z is the output signal of the FSM, displayed by LEDG0, indicating that the state machine is in state E or state I.

SW1 is connected to input w. SW0 is connected to the Reset function, which resets the FSM to the initial state A when the Reset signal is activated. Key0 is connected to the Clock signal for synchronizing state transitions. This ensures that the state transitions occur sequentially during each clock cycle.

3. Design Entry

(1) virtual finite state machine

FSM_logic.vhd(one-hot codes):

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FSM_logic is
    Port ( clk : in STD_LOGIC;
           reset : in STD_LOGIC;
           w : in STD_LOGIC;
           z : out STD_LOGIC;
           y : buffer STD_LOGIC_VECTOR(8 downto 0) := "000000001";
           LEDR : out STD_LOGIC_VECTOR(8 downto 0));
end FSM_logic;

architecture Behavioral of FSM_logic is
begin
    process(clk)
    begin
        if rising_edge(clk) then
            y(8) <= (y(7) or y(8)) and w and reset; --I
            y(7) <= y(6) and w and reset; --H
            y(6) <= y(5) and w and reset; --G
            y(5) <= (y(0) or y(1) or y(2) or y(3) or y(4))and w and reset; --F
            y(4) <= (y(3) or y(4)) and not w and reset; --E
            y(3) <= y(2) and not w and reset; --D
            y(2) <= y(1) and not w and reset; --C
            y(1) <= (y(0) or y(5) or y(6) or y(7) or y(8))and not w and reset; --B
            y(0) <= not reset; --A
        end if;
    end process;

    -- Output Z logic
    z <= y(4) or y(8); -- E and I are the accepting states

    -- LED output to show current state
    LEDR <= y;
end Behavioral;
```

FSM_logic.vhd(modified one-hot codes):

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FSM_logic_modified is
    Port ( clk : in STD_LOGIC;
           reset : in STD_LOGIC;
           w : in STD_LOGIC;
           z : out STD_LOGIC;
           y : buffer STD_LOGIC_VECTOR(8 downto 0) := "000000000";
           LEDR : out STD_LOGIC_VECTOR(8 downto 0));
end FSM_logic_modified;

architecture Behavioral of FSM_logic_modified is
begin
    -- Next state logic
    process(clk)
    begin
        if rising_edge(clk) then
            y(8) <= (y(7) or y(8)) and w and reset; --I
            y(7) <= y(6) and w and reset; --H
            y(6) <= y(5) and w and reset; --G
            y(5) <= (not y(0) or y(1) or y(2) or y(3) or y(4))and w and reset; --F
            y(4) <= (y(3) or y(4)) and not w and reset; --E
            y(3) <= y(2) and not w and reset; --D
            y(2) <= y(1) and not w and reset; --C
            y(1) <= (not y(0) or y(5) or y(6) or y(7) or y(8))and not w and reset; --B
            y(0) <= reset; --A
        end if;
    end process;

    -- Output Z logic
    z <= y(4) or y(8); -- E and I are the accepting states

    -- LED output to show current state
    LEDR <= y;
end Behavioral;
```

(2) finite state machine

FSM_state.vhd:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FSM_state is
    Port ( clk : in STD_LOGIC;
           reset : in STD_LOGIC;
           w : in STD_LOGIC;
           z : out STD_LOGIC;
           next_state_leds: buffer STD_LOGIC_VECTOR(8 downto 0);
           state_leds : out STD_LOGIC_VECTOR(8 downto 0));
end FSM_state;

architecture Behavioral of FSM_state is
    type state_type is (A, B, C, D, E, F, G, H, I);
    signal state : state_type := A;
    signal next_state : state_type;
begin
    -- State register
```

```
    process(clk, reset)
    begin
        if rising_edge(clk) then
            if reset = '0' then
                state <= A;
            else
                state <= next_state
            end if;
        end if;
    end process;
```

```
    -- Next state logic
    process(state, w)
    begin
        case state is
            when A =>
                if w = '0' then next_state <= B; else next_state <= F; end if;
                z <= '0';
            when B =>
                if w = '0' then next_state <= C; else next_state <= F; end if;
                z <= '0';
            when C =>
                if w = '0' then next_state <= D; else next_state <= F; end if;
                z <= '0';
            when D =>
                if w = '0' then next_state <= E; else next_state <= F; end if;
                z <= '0';
            when E =>
                if w = '0' then next_state <= E; else next_state <= F; end if;
                z <= '1';
            when F =>
                if w = '1' then next_state <= G; else next_state <= B; end if;
                z <= '0';
            when G =>
                if w = '1' then next_state <= H; else next_state <= B; end if;
                z <= '0';
            when H =>
                if w = '1' then next_state <= I; else next_state <= B; end if;
                z <= '0';
            when I =>
                if w = '1' then next_state <= I; else next_state <= B; end if;
                z <= '1';
        end case;
    end process;
```

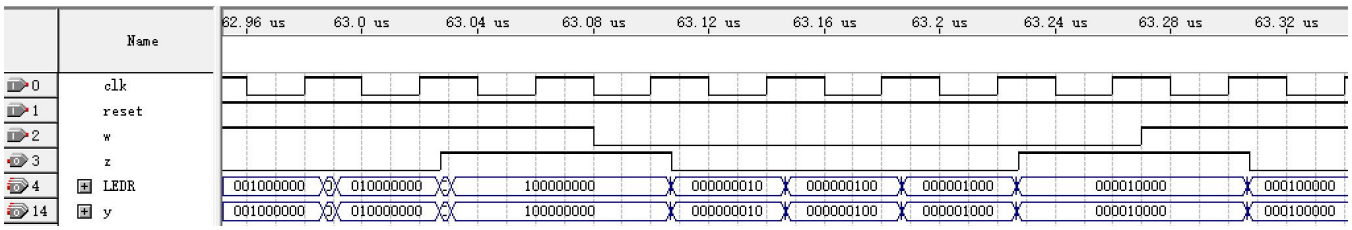
```
66      -- State LEDs
67      process(state, next_state)
68      begin
69          case state is
70              when A => state_leds <= "000000000";
71              when B => state_leds <= "000000011";
72              when C => state_leds <= "000000101";
73              when D => state_leds <= "000001001";
74              when E => state_leds <= "000010001";
75              when F => state_leds <= "000100001";
76              when G => state_leds <= "001000001";
77              when H => state_leds <= "010000001";
78              when I => state_leds <= "100000001";
79          end case;
80
81          case next_state is
82              when A => next_state_leds <= "000000000";
83              when B => next_state_leds <= "000000011";
84              when C => next_state_leds <= "000000101";
85              when D => next_state_leds <= "000001001";
86              when E => next_state_leds <= "000010001";
87              when F => next_state_leds <= "000100001";
88              when G => next_state_leds <= "001000001";
89              when H => next_state_leds <= "010000001";
90              when I => next_state_leds <= "100000001";
91          end case;
92      end process;
93
94  end Behavioral;
```

4. Simulation and Synthesis Results

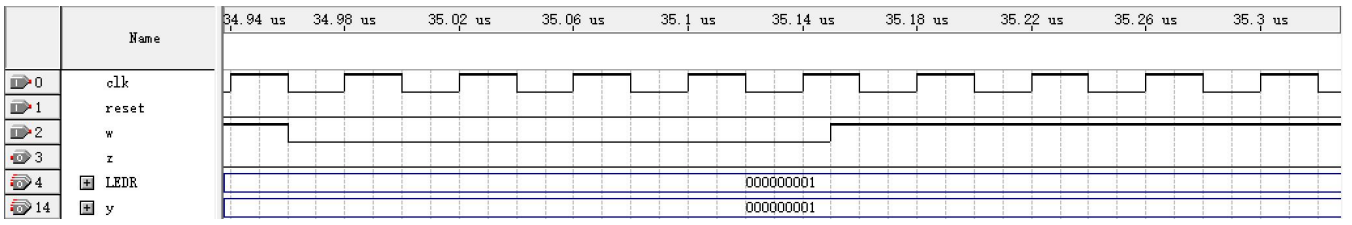
(1)virtual finite state machine

One-hot codes:

Clock on effect:

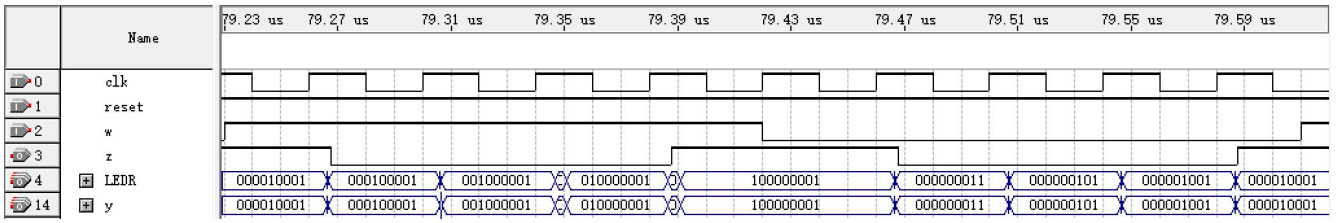


Reset on effect:

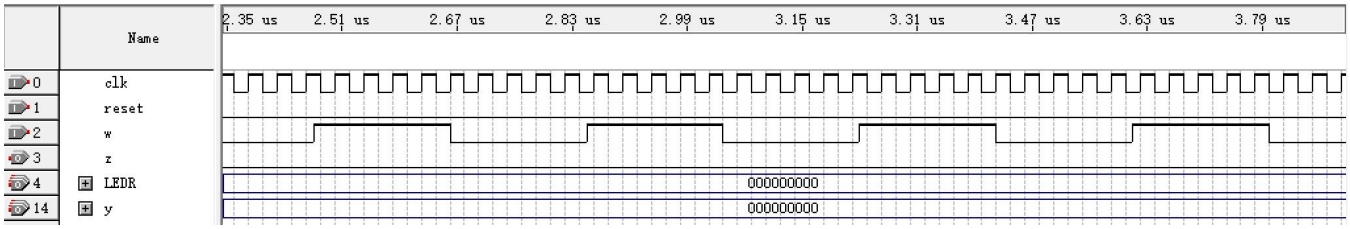


Modified one-hot codes:

Clock on effect:

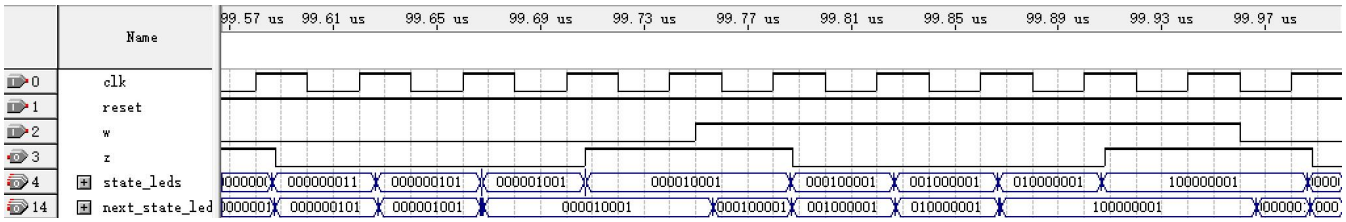


Reset on effect:

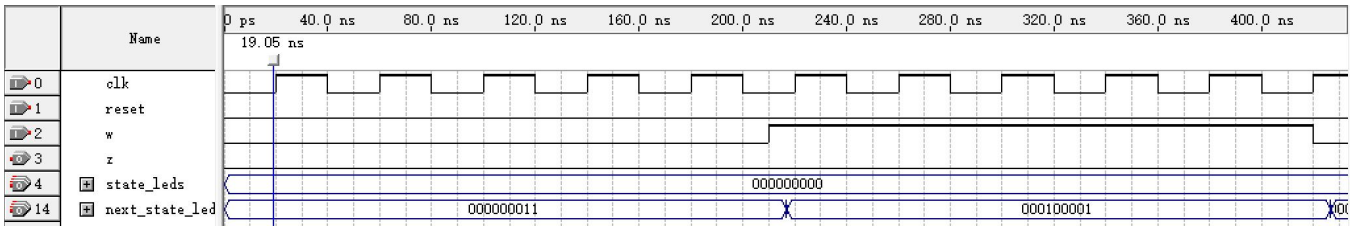


(2)finite state machine

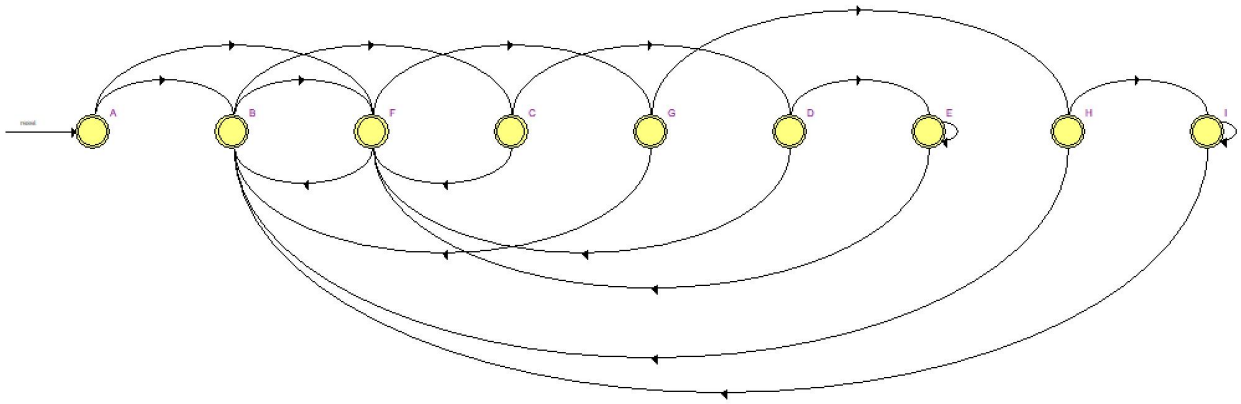
Clock on effect:



Reset on effect:



RTL viewer:

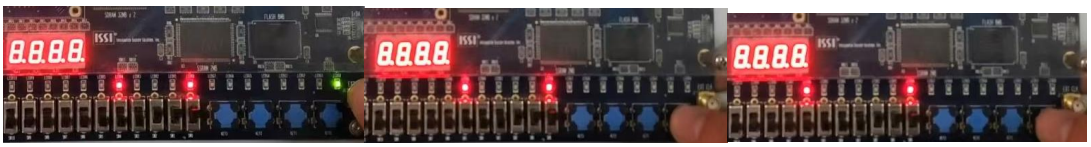


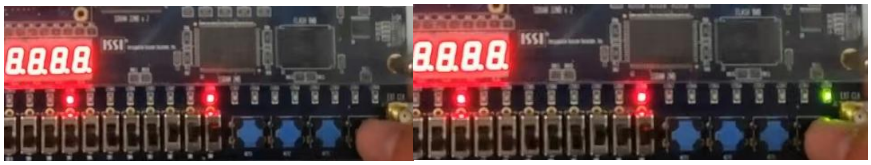
5. Experimental Results

Part (2) is proved in experiment on board.

(2)finite state machine

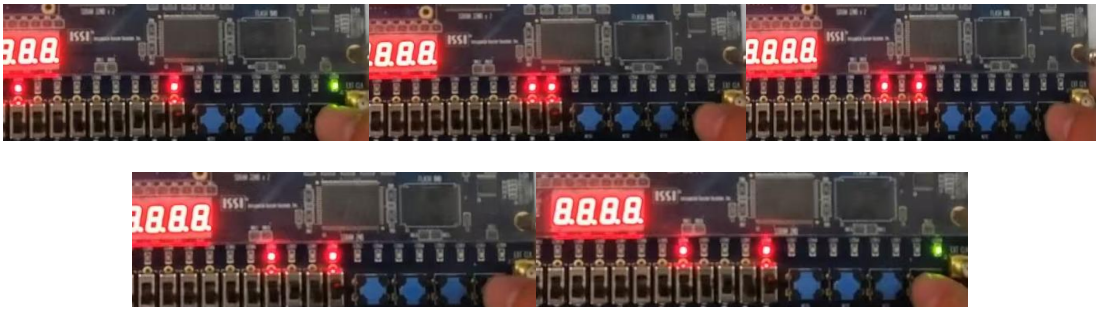
Keep input w as “1” and reset as “1”. Set current state as “E”. Pic.2.2 shows the process of state transition from “E” to “I” every time Key0 is pressed.





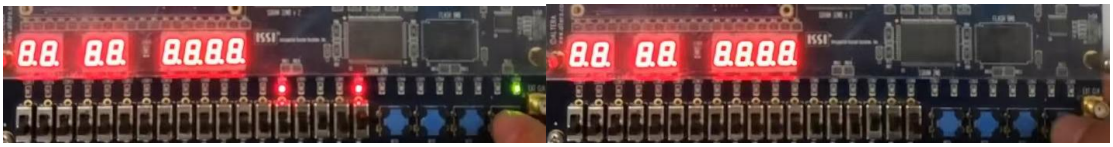
Pic.2.2 state transition(E->F->G->H-> I)

Keep input w as “0” and reset as “1”. Set current state as “I”. Pic.2.3 shows the process of state transition from “I” to “E” every time Key0 is pressed .



Pic.2.3 state transition(I->B->C->D->E)

Set reset as “1”. Pic.2.4 shows the process of state transition to “A” when Key0 is pressed.



Pic.2.4 state transition(E->A)

6. Discussion and Conclusion

In programming part (1) virtual finite state machine, I misunderstood the logic expression as breakdown of the one-hot codes and used next_y as part (2). As a solution, I avoided conditional statement and use only assigning statement.

By applying the theory of FSM to actual VHDL coding, I gained a deeper understanding of the concepts of digital logic design, including the design of state coding, state transfer and output logic. And I learned the importance of ensuring system to be properly reset and initialized to a known state and the critical role of the clock signal in synchronizing a state machine.

While implementing the FSM, I encountered some challenges such as states not updating or incorrect outputs. Through debugging, I learned how to use simulation tools to observe signal changes and troubleshoot problems step by step.